Laboratorio 9 — Hoja de trabajo

Repositorio, paginación, filtros y socios con sus productos

Nombre: Ronald Salvatierra Flores **Fecha:** 15 / 10 / 2026

Comandos: `docker compose up -d --build --wait` · `docker compose exec -T php php verificar.php` (objetivo: 14 de 14; el verificador carga 36 productos y 8 socios). El sitio queda en http://127.0.0.1:8000/productos.

Parte A — Del SQL suelto al repositorio

1. Compara ProductoConsultas (lab 8) con ProductoRepositorioMysql. ¿Qué devuelve cada uno: arreglos u objetos?

ProductoConsultas (lab 8) devuelve arreglos asociativos: las filas tal como salen de MySQL ($fila['precio']). ProductoRepositorioMysql devuelve objetos: Producto (con su Socio dentro) y Pagina. Con objetos los tipos están garantizados (stock es int, categoria es un enum) y el editor y PHP detectan errores de nombres.

2. ¿Qué hace el método privado hidratar()?

Convierte una fila de MySQL en un objeto: crea el Socio con las columnas socio_id, socio_nombre, comunidad y telefono, convierte categoria en el enum Categoria::from(...), pasa stock e id a int y deja precio como texto exacto ("45.00"). Es el único lugar que conoce la forma de las filas.

3. ¿Por qué el controlador recibe un ProductoRepositorio (interfaz) y no un ProductoRepositorioMysql?

Por inversión de dependencias: el controlador solo necesita saber qué se puede hacer (buscar, guardar, eliminar), no cómo se hace. Así se puede cambiar el almacenamiento (otra base, una API, memoria) sin tocar el controlador, y se puede probar con una implementación falsa.

4. Mira ProductoRepositorioMemoria. ¿En qué situación real te serviría tener dos implementaciones?

Para pruebas rápidas sin base de datos (la prueba "Interfaz" de verificar.php), para demostraciones sin Docker y, en la vida real, para cambiar de motor (MySQL → PostgreSQL), usar una API externa o poner una caché delante sin reescribir la aplicación.

Parte B — Paginación (TODO 1 y 2)

1. Abre /productos antes de los TODO. ¿Cuántas filas muestra? ¿Qué dice la paginación?

Con el código de inicio (sin LIMIT y con totalPaginas() fijo) la lista mostraba las 36 filas a la vez, con "Mostrando 1–36 de 36" y "Página 1 de 1". Con los TODO resueltos muestra 10 filas, "Mostrando 1–10 de 36" (Figura 2).

2. TODO 1: ¿cuántas páginas hay con 36 productos de 10 en 10? ¿Y con 0? ¿Por qué ceil y no round?

```
public function totalPaginas(): int
{
    return max(1, (int) ceil($this->total / $this->porPagina));
}
```

36 de 10 en 10 → 4 páginas. 0 productos → 1 página (max(1, …) asegura que exista siempre la página 1, con el mensaje "Sin resultados"). ceil redondea hacia arriba: con 31 productos hay 3,1 páginas y ceil da 4 (la cuarta tiene 1 producto), mientras que round(3.1) daría 3 y ese producto no se podría ver nunca.

3. TODO 2: completa LIMIT :limite OFFSET :desde. Para la página 3, ¿qué valores reciben :limite y :desde?

```
$c = $this->bd->prepare(self::SELECCION . $donde . " ORDER BY $ordenSql LIMIT :limite
OFFSET :desde");
$c->bindValue(':limite', $porPagina, PDO::PARAM_INT);
$c->bindValue(':desde', ($pagina - 1) * $porPagina, PDO::PARAM_INT);
```

OFFSET = (página − 1) × porPágina. En la página 3 con 10 por página: :limite = 10 y :desde = 20 (se saltan 20 filas y se traen 10). En la página 4, :desde = 30 y quedan 6 productos (Figura 3).

4. ¿Por qué se usa bindValue(..., PDO::PARAM_INT) y no se pasan en el arreglo de execute()?

Con execute([...]) PDO envía los valores como texto ('10') y MySQL espera un entero en LIMIT/OFFSET; según la configuración eso produce un error de sintaxis (1064). bindValue con PARAM_INT indica explícitamente que son enteros y funciona siempre, sin depender de la opción EMULATE_PREPARES.

5. ¿Para qué sirve la consulta SELECT COUNT(*) adicional?

LIMIT solo trae la página pedida, así que no se sabe cuántos resultados hay en total. El COUNT(*), con los mismos filtros, da el total que se usa para calcular totalPaginas() y el texto "Mostrando 11–20 de 36".

Parte C — Orden seguro y parámetros de la URL (TODO 3 y 4)

1. Antes de los TODO: ¿qué pasa con ?orden=stock_desc y con ?orden=p.precio; DROP TABLE productos?

Con el código de inicio, el texto de orden se pegaba dentro del SQL ($ordenSql = str_replace('_desc', ' DESC', "p.$orden")). stock_desc respondía 200 porque, por casualidad, se convertía en "p.stock DESC", que es SQL válido aunque no estaba en la lista. p.precio; DROP TABLE productos respondía 500 y en datos/errores.log quedaba un error SQLSTATE 42000 (1064, error de sintaxis cerca de DROP TABLE). Con una subconsulta como orden=stock,(SELECT SLEEP(0)) la respuesta era 200: una expresión escrita por el atacante sí se ejecutaba.

2. ¿Se borró la tabla? ¿Por qué no? ¿Significa que el código era seguro?

No se borró: SELECT COUNT(*) FROM productos sigue dando 36. Con consultas preparadas reales, prepare() acepta una sola sentencia, así que el ";" produce un error de sintaxis. Pero eso no significa que el código fuera seguro: la subconsulta del punto anterior demuestra que el texto del usuario llegaba al SQL y podía ejecutar expresiones propias (extraer datos, ralentizar la base…).

3. ¿Por qué no se puede escribir ORDER BY :orden con un marcador?

Un marcador representa un valor (un literal), no un nombre de columna. ORDER BY :orden con 'precio' ordena por el texto constante "precio", es decir, no ordena nada. Por eso se usa una lista permitida: a SQL solo llega un fragmento escrito por nosotros.

```
private const ORDEN_SQL = [
    'nombre'      => 'p.nombre ASC',
    'precio'      => 'p.precio ASC, p.nombre ASC',
    'precio_desc' => 'p.precio DESC, p.nombre ASC',
    'stock'       => 'p.stock ASC, p.nombre ASC',
];

$ordenSql = self::ORDEN_SQL[$orden] ?? self::ORDEN_SQL['nombre'];
```

4. TODO 4: ¿qué debe pasar con ?pagina=-5, ?pagina=abc y ?pagina=99?

-5 → página 1 · abc → página 1 · 99 → la última página que exista (Página 4 de 4). También cualquier valor no entero o un arreglo (pagina[]=2) vuelve a la página 1, sin errores 500. El verificador

lo comprueba ("pagina=abc → 1 · pagina=99 → la última (4)"). En el navegador, ?pagina=abc muestra la página 1 (Figura 4).

```
$bruta = $_GET['pagina'] ?? '1';
$pedida = is_string($bruta)
    ? filter_var($bruta, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1, 'default' => 1]])
    : 1;
$pedida = min($pedida, 1_000_000);
```

Parte D — Filtros y relación 1 a N (TODO 5 y 6)

1. TODO 5: filtra por categoría. ¿Cuántos productos de Cera hay? ¿Y de Cera con "vela"?

```
if ($categoria !== null) {
    $condiciones[] = 'p.categoria = :categoria';
    $valores['categoria'] = $categoria->value;
}
```

Hay 10 productos de Cera y 2 de Cera con "vela" (Vela aromática de cera 200 g y Velas de cera (par)). Las mismas condiciones se usan en el COUNT(*) y en el SELECT, por eso el total siempre coincide con las filas.

2. Revisa los enlaces "Anterior"/"Siguiente" con un filtro activo. ¿Qué hace http_build_query? ¿Por qué aparece & en el HTML?

http_build_query arma la cadena de consulta codificando los valores, de modo que los filtros se conservan al cambiar de página. En HTML el carácter & debe escribirse & (la plantilla escapa todo con htmlspecialchars); el navegador lo convierte de nuevo en & al seguir el enlace:

```
href="/productos?categoria=Miel&amp;orden=precio_desc&amp;pagina=2"
```

3. TODO 6: completa deSocio(). Abre /socios/3. ¿Qué tipo de relación hay entre socios y productos?

```
public function deSocio(int $socioId): array
{
    $c = $this->bd->prepare(self::SELECCION . ' WHERE p.socio_id = ? ORDER BY p.nombre');
    $c->execute([$socioId]);
    return array_map($this->hidratar(...), $c->fetchAll());
}
```

/socios/3 (Elena Masabí) muestra 4 productos y "valor en stock Bs 2.847,50". Es una relación uno a muchos (1 a N): un socio tiene muchos productos y cada producto pertenece a un solo socio (clave foránea productos.socio_id → socios.id).

Parte E — Verificación

php verificar.php → objetivo 14 de 14.

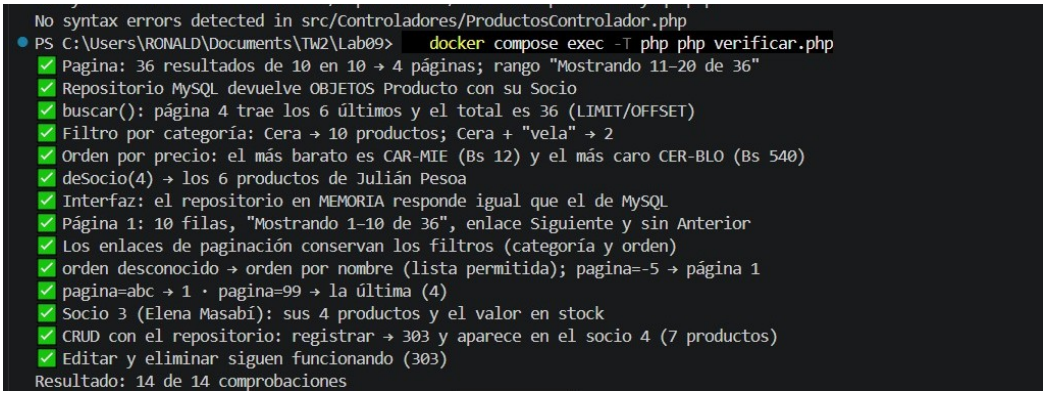


Figura 1. Verificador: 14 de 14 comprobaciones.

Pruebas manuales en el navegador:

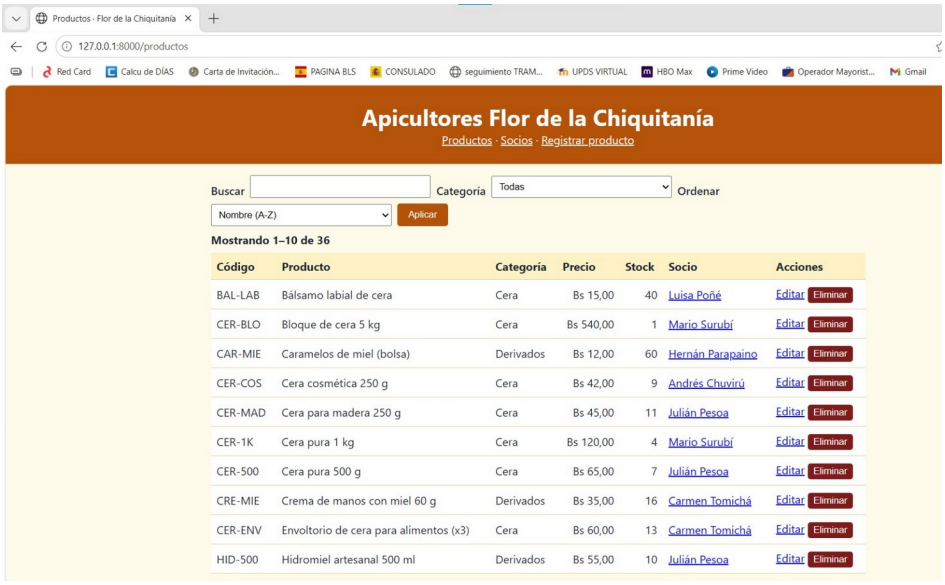


Figura 2. /productos: primera página, "Mostrando 1–10 de 36".



Figura 3. /productos?pagina=4: "Mostrando 31–36 de 36" (6 filas), "Página 4 de 4".

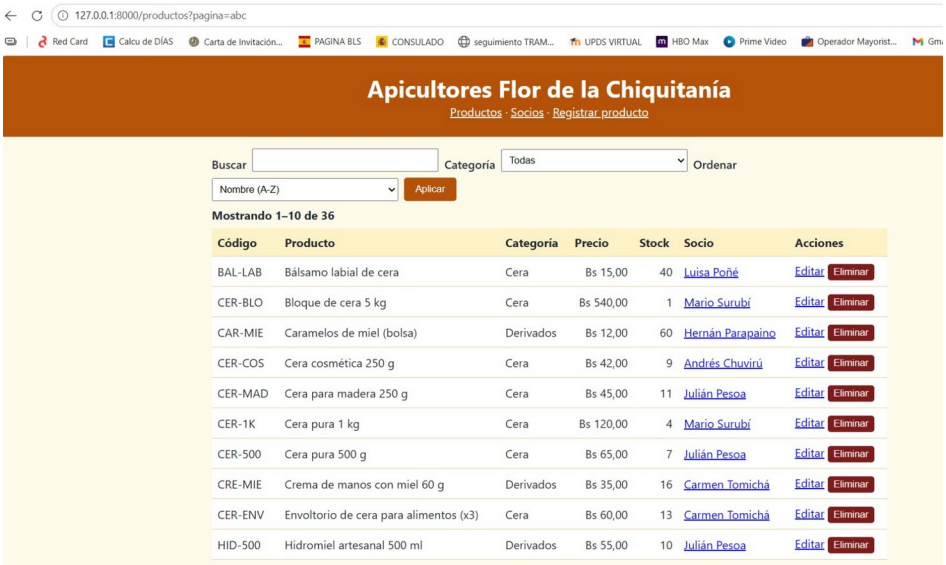


Figura 4. /productos?pagina=abc: la página inválida vuelve a la página 1.

Reflexión

Si mañana la asociación decide usar PostgreSQL o una API externa en lugar de MySQL, ¿qué archivos cambiarías y cuáles no? Justifica.

Cambiarían: una nueva implementación de ProductoRepositorio (por ejemplo ProductoRepositorioPostgres o ProductoRepositorioApi) con su propio SQL o sus llamadas HTTP, adaptando el dialecto (LIKE → ILIKE en PostgreSQL; el error de duplicado pasa del 1062 de MySQL al SQLSTATE 23505). También config/bd.php y Conexion.php (el DSN pgsql:), el Dockerfile (pdo_pgsql) y el docker-compose.yml (el servicio de base de datos). Y public/index.php, que es el único lugar donde se elige la implementación (new ProductoRepositorioMysql(...)).

No cambiarían: ProductosControlador, ValidadorProducto, Pagina, los modelos (Producto, Socio, Categoria), las plantillas, el Router ni verificar.php (la prueba "Interfaz" ya demuestra que otra implementación responde igual). La razón es que todos dependen de la interfaz o de objetos, nunca de MySQL. Una salvedad: SocioRepositorio es hoy una clase concreta con PDO; para independizarla por completo habría que extraerle también una interfaz, como se hizo con los productos.

Anexo — Archivos completados

Archivos de la carpeta inicio con los TODO resueltos. Los demás archivos no se modificaron, salvo docker-compose.yml (nombre de la base de datos corregido a apicultores y name: tw2-lab09).

src/Repositorios/Pagina.php

```
<?php
// src/Repositorios/Pagina.php — Un "trozo" de resultados + lo necesario para dibujar la paginación.
declare(strict_types=1);

namespace Apicultores\Repositorios;

final readonly class Pagina
{
    /** @param array $elementos los elementos de ESTA página */
    public function __construct(
        public array $elementos,
        public int $total,          // total de resultados (todas las páginas)
        public int $numero,         // página actual, desde 1
        public int $porPagina,
    ) {
    }

    public function totalPaginas(): int
    {
        // TODO 1: total de páginas = total / porPagina redondeado hacia ARRIBA (ceil), y como mínimo 1.
        //         36 resultados de 10 en 10 → 4 páginas; 0 resultados → 1 página.
        return max(1, (int) ceil($this->total / $this->porPagina));
    }

    public function tieneAnterior(): bool
    {
        return $this->numero > 1;
    }

    public function tieneSiguiente(): bool
    {
        return $this->numero < $this->totalPaginas();
    }

    /** "Mostrando 11–20 de 36" */
    public function rango(): string
    {
        if ($this->total === 0) {
            return 'Sin resultados';
        }
        $desde = ($this->numero - 1) * $this->porPagina + 1;
        return sprintf('Mostrando %d–%d de %d', $desde, $desde + count($this->elementos) - 1, $this-
>total);
    }
}
```

src/Repositorios/ProductoRepositorioMysql.php

```
<?php
```

```
// src/Repositorios/ProductoRepositorioMysql.php — Implementación con MySQL + PDO. TODO el SQL de
productos vive aquí.
declare(strict_types=1);

namespace Apicultores\Repositorios;

use Apicultores\Excepciones\Conflicto;
use Apicultores\Excepciones\NoEncontrado;
use Apicultores\Modelo\Categoria;
use Apicultores\Modelo\Producto;
use Apicultores\Modelo\Socio;
use PDO;
use PDOException;

final class ProductoRepositorioMysql implements ProductoRepositorio
{
    private const SELECCION = 'SELECT p.id, p.codigo, p.nombre, p.categoria, p.precio, p.stock,
                                      s.id AS socio_id, s.nombre AS socio_nombre, s.comunidad,
s.telefono
                               FROM productos p
                               JOIN socios s ON s.id = p.socio_id';

    /** Lista permitida: clave pública → fragmento SQL. Los NOMBRES de columnas no pueden ir como
marcadores. */
    private const ORDEN_SQL = [
        'nombre' => 'p.nombre ASC',
        'precio' => 'p.precio ASC, p.nombre ASC',
        'precio_desc' => 'p.precio DESC, p.nombre ASC',
        'stock' => 'p.stock ASC, p.nombre ASC',
    ];

    public function __construct(private readonly PDO $bd)
    {
    }

    public function porCodigo(string $codigo): ?Producto
    {
        $c = $this->bd->prepare(self::SELECCION . ' WHERE p.codigo = ?');
        $c->execute([$codigo]);
        $fila = $c->fetch();
        return $fila ? $this->hidratar($fila) : null;
    }

    public function buscar(string $texto, ?Categoria $categoria, string $orden, int $pagina, int
$porPagina): Pagina
    {
        // 1) Filtros: se arman las condiciones con marcadores y sus valores por separado
        $condiciones = ['(p.nombre LIKE :patron OR p.codigo LIKE :patron2)'];
        $patron = '%' . addcslashes($texto, '%_\\') . '%';
        $valores = ['patron' => $patron, 'patron2' => $patron];
        if ($categoria !== null) {
            $condiciones[] = 'p.categoria = :categoria';
            $valores['categoria'] = $categoria->value;
        }
        $donde = ' WHERE ' . implode(' AND ', $condiciones);
```

```
        // 2) Total (para saber cuántas páginas hay)
        $c = $this->bd->prepare('SELECT COUNT(*) FROM productos p' . $donde);
        $c->execute($valores);
        $total = (int) $c->fetchColumn();

        // 3) Solo la página pedida: LIMIT/OFFSET como ENTEROS (bindValue con PDO::PARAM_INT)
        // Lista permitida: a SQL solo llega un fragmento ESCRITO POR NOSOTROS, nunca el texto del
usuario.
        $ordenSql = self::ORDEN_SQL[$orden] ?? self::ORDEN_SQL['nombre'];
        $c = $this->bd->prepare(self::SELECCION . $donde . " ORDER BY $ordenSql LIMIT :limite
OFFSET :desde");
        foreach ($valores as $nombre => $valor) {
            $c->bindValue(':' . $nombre, $valor);                              // textos:
PDO::PARAM_STR (por defecto)
        }
        $c->bindValue(':limite', $porPagina, PDO::PARAM_INT);                  // LIMIT/OFFSET exigen
ENTEROS, no '10'
        $c->bindValue(':desde', ($pagina - 1) * $porPagina, PDO::PARAM_INT);
        $c->execute();

        return new Pagina(array_map($this->hidratar(...), $c->fetchAll()), $total, $pagina, $porPagina);
    }

    public function deSocio(int $socioId): array
    {
        $c = $this->bd->prepare(self::SELECCION . ' WHERE p.socio_id = ? ORDER BY p.nombre');
        $c->execute([$socioId]);
        return array_map($this->hidratar(...), $c->fetchAll());
    }

    public function existeCodigo(string $codigo): bool
    {
        $c = $this->bd->prepare('SELECT 1 FROM productos WHERE codigo = ?');
        $c->execute([$codigo]);
        return $c->fetchColumn() !== false;
    }

    public function guardar(array $d): void
    {
        $sql = $this->existeCodigo($d['codigo'])
            ? 'UPDATE productos SET nombre = :nombre, categoria = :categoria, precio = :precio, stock
= :stock, socio_id = :socio_id
               WHERE codigo = :codigo'
            : 'INSERT INTO productos (codigo, nombre, categoria, precio, stock, socio_id)
               VALUES (:codigo, :nombre, :categoria, :precio, :stock, :socio_id)';
        try {
            $this->bd->prepare($sql)->execute(array_intersect_key($d, array_flip(['codigo', 'nombre',
'categoria', 'precio', 'stock', 'socio_id'])));
        } catch (PDOException $e) {
            if (($e->errorInfo[1] ?? null) === 1062) {
                throw new Conflicto("Ya existe un producto con el código {$d['codigo']}.", 0, $e);
            }
            throw $e;
        }
    }
```

```
    public function eliminar(string $codigo): void
    {
        $c = $this->bd->prepare('DELETE FROM productos WHERE codigo = ?');
        $c->execute([$codigo]);
        if ($c->rowCount() === 0) {
            throw new NoEncontrado("No existe el producto $codigo.");
        }
    }

    /** Fila de MySQL → objeto Producto (con su Socio). */
    private function hidratar(array $f): Producto
    {
        return new Producto(
            $f['codigo'], $f['nombre'], Categoria::from($f['categoria']), $f['precio'], (int)
$f['stock'],
            new Socio((int) $f['socio_id'], $f['socio_nombre'], $f['comunidad'], $f['telefono']),
            (int) $f['id'],
        );
    }
}
```

src/Controladores/ProductosControlador.php

```
<?php
// src/Controladores/ProductosControlador.php — Usa REPOSITORIOS (interfaces) en lugar de SQL. Pagina,
filtra y ordena.
declare(strict_types=1);

namespace Apicultores\Controladores;

use Apicultores\Excepciones\NoEncontrado;
use Apicultores\Modelo\Categoria;
use Apicultores\Modelo\Producto;
use Apicultores\Repositorios\ProductoRepositorio;
use Apicultores\Repositorios\SocioRepositorio;
use Apicultores\Validacion\ValidadorProducto;
use Apicultores\Vista\Plantilla;

final class ProductosControlador
{
    public const POR_PAGINA = 10;

    public function __construct(
        private readonly ProductoRepositorio $productos,
        private readonly SocioRepositorio $socios,
        private readonly ValidadorProducto $validador,
        private readonly Plantilla $vista,
    ) {
    }

    public function listar(): string
    {
        // Todo lo que llega por GET se normaliza: texto, categoría válida o null, orden permitido,
página >= 1
        $q = $this->texto('q');
        $categoria = Categoria::tryFrom($this->texto('categoria'));
        $orden = $this->texto('orden');
```

```
        if (!array_key_exists($orden, ProductoRepositorio::ORDENES)) {
            $orden = 'nombre';                                          // orden desconocido → el de por
defecto
        }
        $bruta = $_GET['pagina'] ?? '1';                                // puede llegar como arreglo (?
pagina[]=2)
        $pedida = is_string($bruta)
            ? filter_var($bruta, FILTER_VALIDATE_INT, ['options' => ['min_range' => 1, 'default' => 1]])
            : 1;
        $pedida = min($pedida, 1_000_000);                              // evita OFFSET absurdos; luego
se ajusta a la última

        $pagina = $this->productos->buscar($q, $categoria, $orden, $pedida, self::POR_PAGINA);
        if ($pedida > $pagina->totalPaginas()) {              // ?pagina=99 → la última página que
exista
            $pagina = $this->productos->buscar($q, $categoria, $orden, $pagina->totalPaginas(),
self::POR_PAGINA);
        }

        $filtros = array_filter(['q' => $q, 'categoria' => $categoria?->value, 'orden' => $orden ===
'nombre' ? null : $orden]);
        $enlace = fn(int $n): string => '/productos?' . http_build_query($filtros + ['pagina' => $n]);
        return $this->vista->pagina('productos', [
            'q' => $q, 'categoria' => $categoria?->value ?? '', 'orden' => $orden,
            'categorias' => array_map(fn(Categoria $c): string => $c->value, Categoria::cases()),
            'ordenes' => ProductoRepositorio::ORDENES,
            'productos' => array_map($this->fila(...), $pagina->elementos),
            'rango' => $pagina->rango(), 'numero' => $pagina->numero, 'totalPaginas' => $pagina-
>totalPaginas(),
            'anterior' => $pagina->tieneAnterior() ? $enlace($pagina->numero - 1) : null,
            'siguiente' => $pagina->tieneSiguiente() ? $enlace($pagina->numero + 1) : null,
            'mensaje' => $this->mensaje(),
        ], 'Productos');
    }

    public function socios(): string
    {
        return $this->vista->pagina('socios', [
            'socios' => array_map(fn($s): array => ['id' => $s->id, 'nombre' => $s->nombre, 'comunidad'
=> $s->comunidad],
                $this->socios->todos()),
        ], 'Socios');
    }

    public function socio(string $id): string
    {
        $socio = $this->socios->porId((int) $id) ?? throw new NoEncontrado('No existe ese socio.');
        $productos = $this->productos->deSocio($socio->id);
        return $this->vista->pagina('socio', [
            'nombre' => $socio->nombre, 'comunidad' => $socio->comunidad, 'telefono' => $socio->telefono
?? 'sin teléfono',
            'productos' => array_map($this->fila(...), $productos),
            'valor' => bs(array_sum(array_map(fn(Producto $p): float => (float) $p->precio * $p->stock,
$productos))),
        ], $socio->nombre);
    }
```

```
    public function nuevo(): string
    {
        return $this->formulario('/productos', [], [], true);
    }

    public function crear(): never
    {
        $this->productos->guardar($this->validador->validar($_POST, nuevo: true));
        $this->redirigir('/productos?creado=' . urlencode(strtoupper(trim((string) $_POST['codigo']))));
    }

    public function editar(string $codigo): string
    {
        $p = $this->productos->porCodigo($codigo) ?? throw new NoEncontrado("No existe el producto
$codigo.");
        return $this->formulario('/productos/' . rawurlencode($codigo), [
            'codigo' => $p->codigo, 'nombre' => $p->nombre, 'categoria' => $p->categoria->value,
            'precio' => $p->precio, 'stock' => $p->stock, 'socio_id' => $p->socio->id], [], false);
    }

    public function actualizar(string $codigo): never
    {
        $this->productos->porCodigo($codigo) ?? throw new NoEncontrado("No existe el producto
$codigo.");
        $this->productos->guardar($this->validador->validar(['codigo' => $codigo] + $_POST, nuevo:
false));
        $this->redirigir('/productos?actualizado=' . urlencode($codigo));
    }

    public function eliminar(string $codigo): never
    {
        $this->productos->eliminar($codigo);
        $this->redirigir('/productos?eliminado=' . urlencode($codigo));
    }

    public function formulario(string $accion, array $datos, array $errores, bool $nuevo): string
    {
        $campos = ['codigo', 'nombre', 'categoria', 'precio', 'stock', 'socio_id'];
        $socios = [];
        foreach ($this->socios->todos() as $s) {
            $socios[$s->id] = $s->nombre;
        }
        return $this->vista->pagina('formulario', [
            'accion' => $accion, 'nuevo' => $nuevo, 'errores' => $errores,
            'datos' => array_map(fn(mixed $v): string => is_scalar($v) ? (string) $v : '',
array_intersect_key($datos, array_flip($campos)) + array_fill_keys($campos, '')),
            'categorias' => array_map(fn(Categoria $c): string => $c->value, Categoria::cases()),
            'socios' => $socios,
        ], $nuevo ? 'Registrar producto' : 'Editar producto');
    }

    private function fila(Producto $p): array
    {
        return ['codigo' => $p->codigo, 'nombre' => $p->nombre, 'categoria' => $p->categoria->value,
'precio_bs' => $p->precioBs(),
```

```
                'stock' => $p->stock, 'socio' => $p->socio->nombre, 'socio_id' => $p->socio->id,
'comunidad' => $p->socio->comunidad];
    }

    private function mensaje(): string
    {
        foreach (['creado' => 'registrado', 'actualizado' => 'actualizado', 'eliminado' => 'eliminado']
as $clave => $verbo) {
            if (is_string($_GET[$clave] ?? null)) {
                return "Producto {$_GET[$clave]} $verbo.";
            }
        }
        return '';
    }

    /** Un parámetro GET como texto: si no llega, o llega como arreglo (?q[]=x), es ''. */
    private function texto(string $clave): string
    {
        $valor = $_GET[$clave] ?? '';
        return is_string($valor) ? trim($valor) : '';
    }

    private function redirigir(string $url): never
    {
        header("Location: $url", true, 303);
        exit;
    }
}
```

docker-compose.yml

```
# docker-compose.yml — Lab 9: PHP 8.4 + MySQL 8.4 LTS (solo DESARROLLO: las claves están a la vista).
#   Iniciar:        docker compose up --build
#   Verificar:      docker compose exec php php verificar.php
#   Consola MySQL:  docker compose exec mysql mysql -u apicultores -pmiel2026 apicultores
#   Datos de cero:  docker compose down -v && docker compose up --build   (¡borra la base!)
#   Si el stack del lab 8 sigue encendido, detenlo antes (usa los mismos puertos 3307 y 8000):  docker
compose down
name: tw2-lab09   # nombre propio del proyecto: no se mezcla con los de otros laboratorios

services:
  mysql:
    image: mysql:8.4
    container_name: apicultor-mysql
    environment:
      MYSQL_DATABASE: apicultores
      MYSQL_USER: apicultores
      MYSQL_PASSWORD: miel2026
      MYSQL_ROOT_PASSWORD: raiz2026
      TZ: America/La_Paz
    ports:
      - "127.0.0.1:3307:3306"   # solo accesible desde tu PC (3307 para no chocar con otro MySQL/XAMPP)
    volumes:
      - ./sql:/docker-entrypoint-initdb.d:ro   # los .sql se ejecutan en orden la PRIMERA vez
      - datos_mysql:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "127.0.0.1", "-u", "root", "-praiz2026"]
```

```
      interval: 5s
      retries: 20

  php:
    build: .
    working_dir: /app
    depends_on:
      mysql:
        condition: service_healthy   # PHP arranca cuando MySQL ya está listo
    environment:
      # Dentro de Docker, MySQL se llama "mysql" (nombre del servicio) y usa su puerto interno 3306
      BD_HOST: mysql
      BD_PUERTO: "3306"
      BD_NOMBRE: apicultores
      BD_USUARIO: apicultores
      BD_CLAVE: miel2026
    volumes:
      - .:/app
    ports:
      - "127.0.0.1:8000:8000"      # abre http://127.0.0.1:8000
    command: sh -c "composer dump-autoload && php -S 0.0.0.0:8000 -t public public/index.php"

volumes:
  datos_mysql:
```